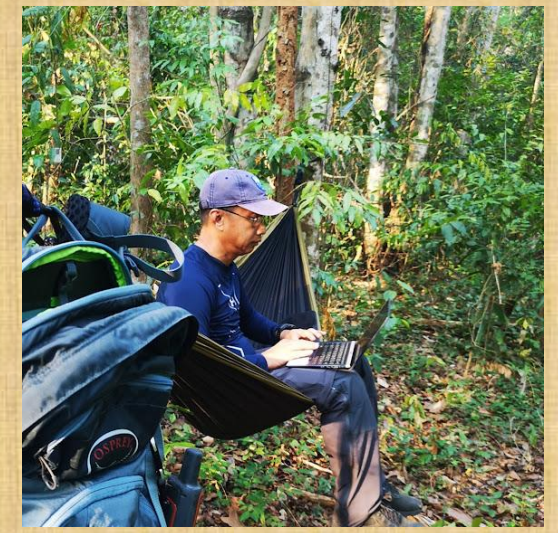
ITCS214 : DATA STRUCTURES & ALGORITHM ANALYSIS

Lecture 2 · Analysis of Algorithms





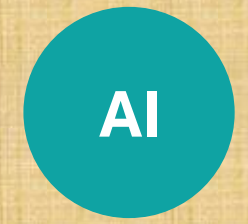
AI can write code. Can it write good code?

Asst. Prof. Dr. Ananta Srisuphab
ananta.sri@mahidol.ac.th
2026/1

Mahidol University
Faculty of Information and Communication Technology

Today's Roadmap

1. Hook
2. Why It Matters
3. Mystery Algorithm
4. Experiments
5. Math That Survives Any Computer
6. Judging AI Code



THE HOOK

Can AI Write Efficient Code?

AI Solution A

Solution A - generated by AI

```
public static int[] twoSumBruteForce(int[] nums, int target) {
    for (int i = 0; i < nums.length; i++) {
        for (int j = i + 1; j < nums.length; j++) {
            if (nums[i] + nums[j] == target) {
                return new int[] { i, j };
            }
        }
    }
    throw new IllegalArgumentException("No solution found");
}
```

AI Solution B

Solution B

```
public static int[] twoSumFast(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (seen.containsKey(complement)) {
            return new int[] { seen.get(complement), i };
        }
        seen.put(nums[i], i);
    }
    throw new IllegalArgumentException("No solution found");
}
```

The Hidden Force Behind Everyday Apps

G

Google Search

Billions of pages searched in milliseconds.

N

Netflix

Recommends from 15,000+ titles, instantly.

GPS

GPS Navigation

Millions of road segments, real-time routing.

AI

ChatGPT

Predicts the next token, every single word.

WHY IT MATTERS

Same hardware. Same data.
Different algorithm.
Different universe of performance.

Efficiency is a property of the algorithm -- not the machine.

2

THINK - PAIR - SHARE (2 MIN)

What do Google, Netflix, GPS, and ChatGPT have in common, in terms of scale?

Think 45s -> Pair 45s -> Share

What does this method do?

? mystery Algorithm

```
public static void mysteryAlgorithm(int[] arr) {
    for (int i = 1; i < arr.length; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

LIVE CODING

Benchmark Harness Setup

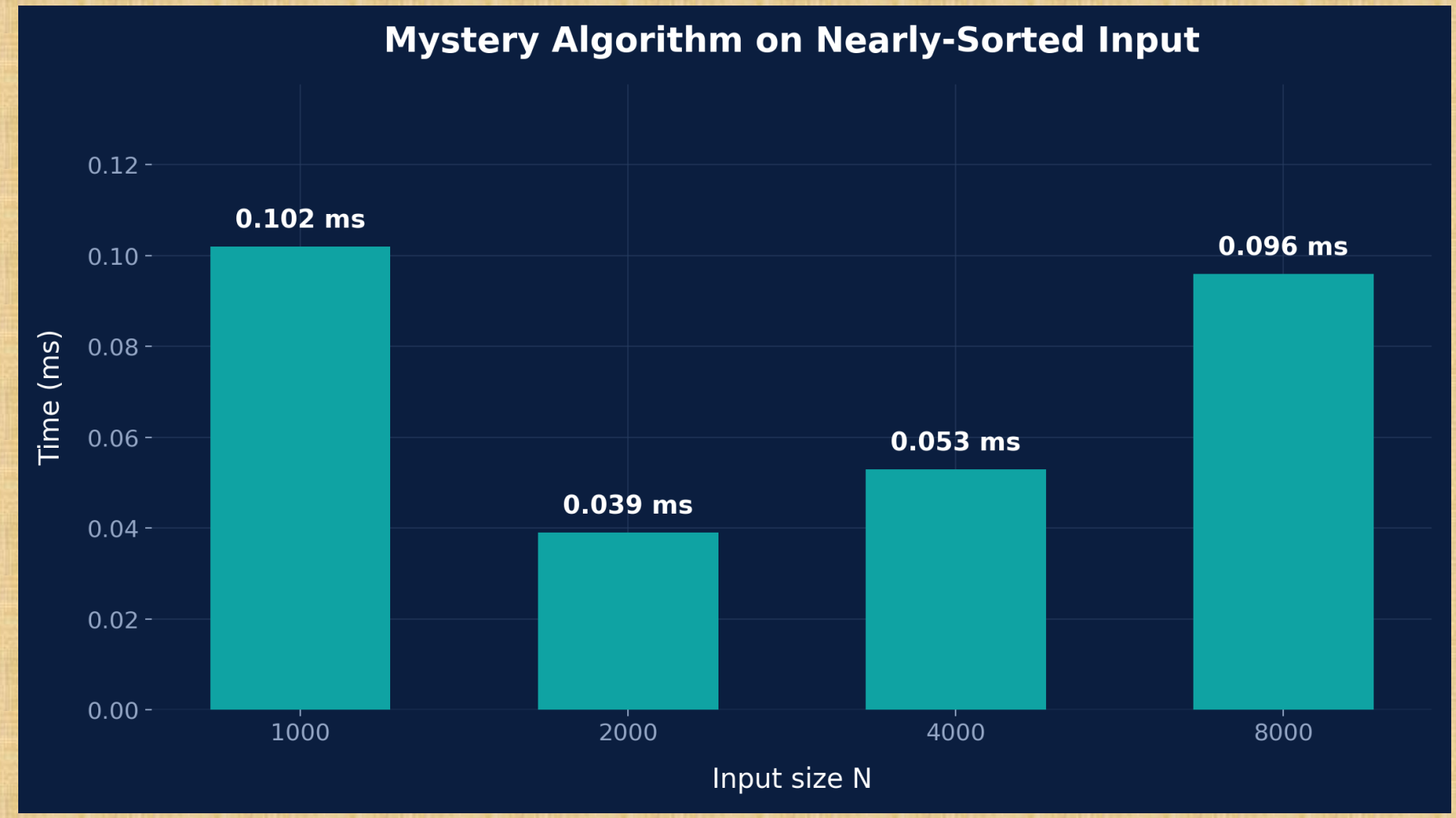
Type this together

```
public static double timeSortMillis(
        Consumer<int[]> sortAlgorithm, int[] input,
        int warmupRuns, int measuredRuns) {

    for (int i = 0; i < warmupRuns; i++) {
        sortAlgorithm.accept(input.clone());
    }

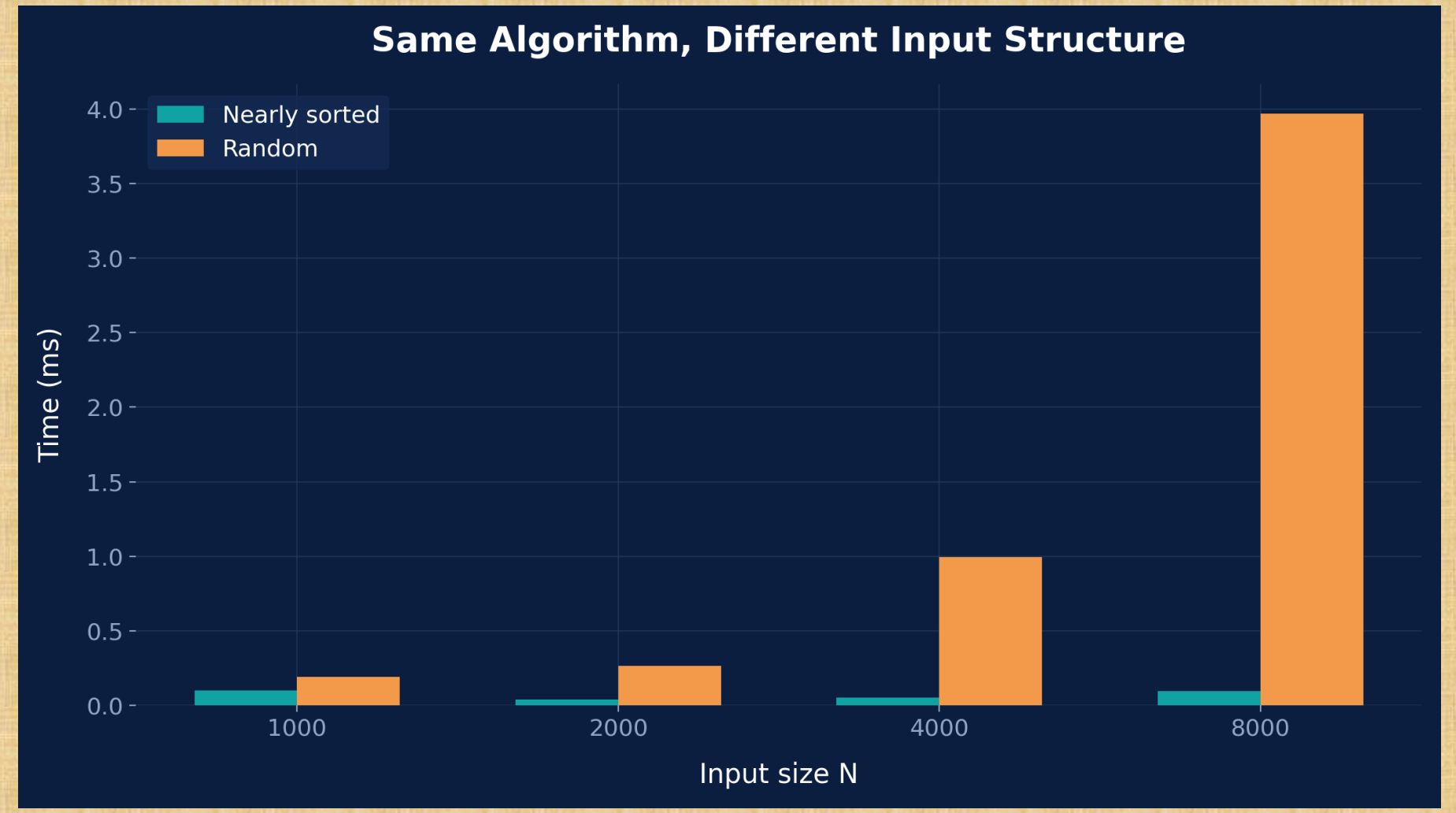
    long totalNanos = 0L;
    for (int i = 0; i < measuredRuns; i++) {
        var copy = input.clone();
        long start = System.nanoTime();
        sortAlgorithm.accept(copy);
        totalNanos += System.nanoTime() - start;
    }
    return (totalNanos / (double) measuredRuns) / 1_000_000.0;
}
```

Mystery Algorithm on Nearly-Sorted Input



Notice anything? Runtime barely grows as N doubles.

Same Algorithm, Different Input Structure



Same algorithm, same sizes -- why is one so much slower?



THE REVEAL

It was Insertion Sort.

Nearly-sorted data needs very few shifts -> fast.
Random data needs many shifts -> slow.



APPROACH #1

Experimental Analysis

Run the code. Measure the clock. Trust the data -- carefully.

System.nanoTime()

Core API

```
long start = System.nanoTime();
// code to measure
long end = System.nanoTime();
double elapsedMillis = (end - start) / 1_000_000.0;

// Never use System.currentTimeMillis() for benchmarking --
// it is for wall-clock time, not precision.
```

The Full Benchmark Class

Benchmark.java

```
public final class Benchmark {
    private Benchmark() {}

    public static double timeSortMillis(Consumer<int[]> sort,
            int[] input, int warmupRuns, int measuredRuns) {
        for (int i = 0; i < warmupRuns; i++) {
            sort.accept(input.clone());
        }
        long total = 0L;
        for (int i = 0; i < measuredRuns; i++) {
            var copy = input.clone();
            long start = System.nanoTime();
            sort.accept(copy);
            total += System.nanoTime() - start;
        }
        return (total / (double) measuredRuns) / 1_000_000.0;
    }
}
```

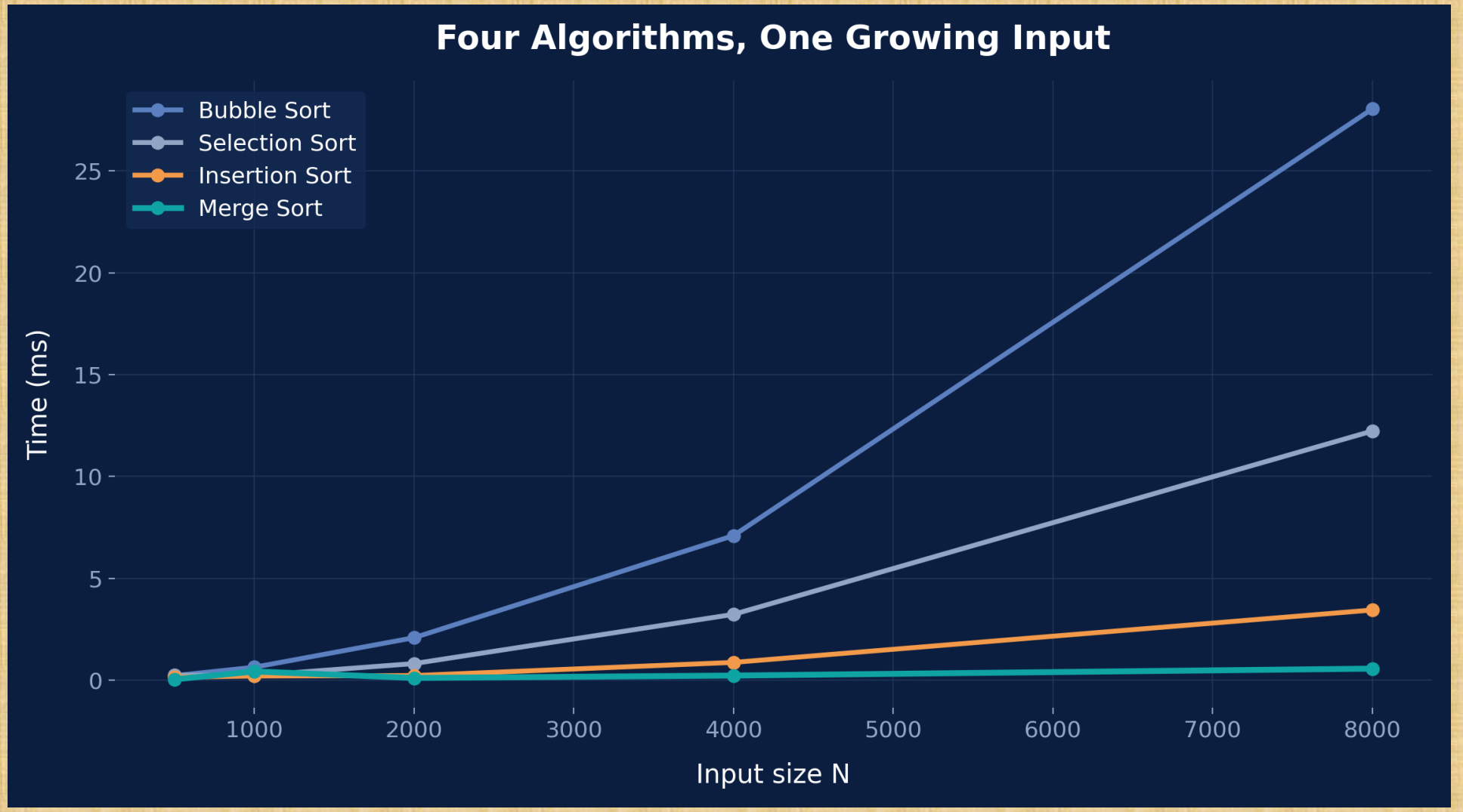
Input Generators: Random vs Reverse

ArrayGenerator.java

```
public static int[] randomArray(int size, long seed) {
    var random = new Random(seed);
    return IntStream.generate(() -> random.nextInt(1_000_000))
                    .limit(size)
                    .toArray();
}

public static int[] reverseSortedArray(int size) {
    return IntStream.rangeClosed(1, size)
                    .map(i -> size - i + 1)
                    .toArray();
}
```

Four Algorithms, One Growing Input



One of these lines bends very differently. Which one worries you at N = 10,000,000?

Same Code. Same Data. Different Numbers.

1

Run 1

142.3 ms

≠

2

Run 2

98.7 ms

Four Reasons Benchmarks Lie To You

JVM

JVM Warm-up

First runs are slower before optimization kicks in.

JIT

JIT Compilation

Hot code gets recompiled into faster machine code mid-run.

GC

Garbage Collection

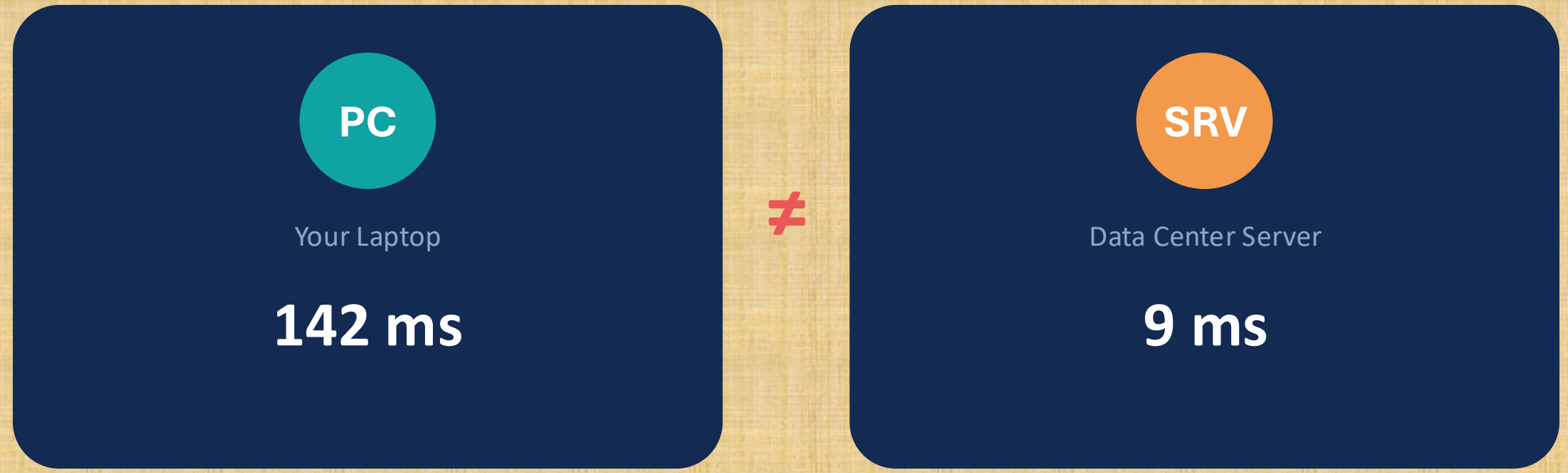
Random pauses to reclaim memory.

CPU

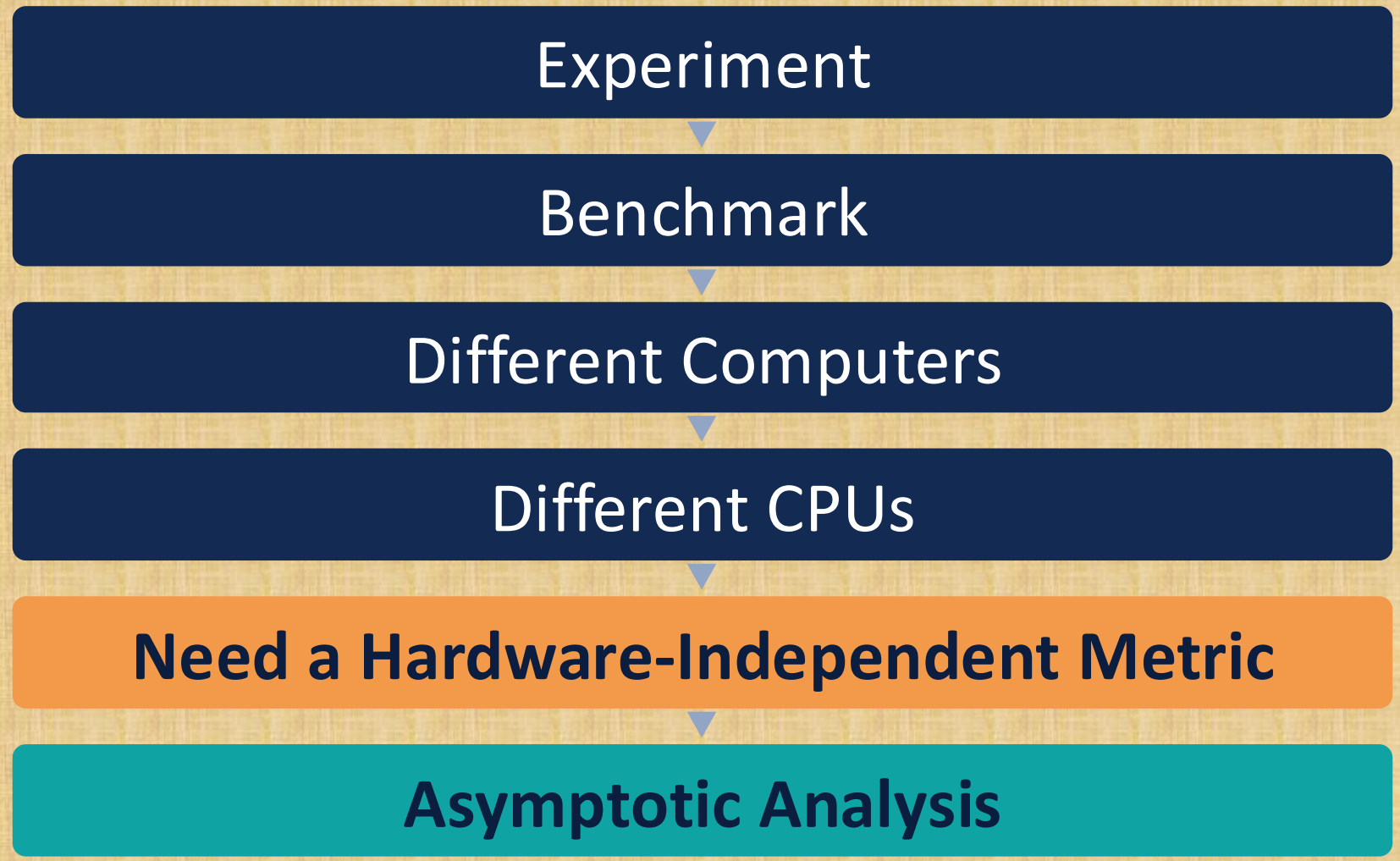
CPU Frequency Scaling

Chips speed up or slow down for power and heat.

Whose Number Is 'The' Running Time?



From Experiment to Asymptotic Analysis



A NEW QUESTION

Instead of 'how many milliseconds?' we ask:

'how does the work grow as N grows?'

What Is a Primitive Operation?

- = Assignment -- storing a value
- < Comparison -- testing a condition
- [] Array Access -- reading/writing an element
- () Method Call -- invoking a method
- → Return -- exiting with a value

findMax() -- Plain

Clean version

```
public static int findMax(int[] arr) {
    int max = arr[0];
    for (int i = 1; i < arr.length; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}
```

findMax() -- Every Operation, Color-Coded

```java
public static int findMax(int[] arr) {
    int max = arr[0];
    for (int i = 1; i < arr.length; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}
```

Assignment Comparison Array Access Return

Click through line by line -- name the operation type out loud for each color.

Operations Per Line

Line	Operation	Times Executed
max = arr[0]	assign + access	1
i = 1	assign	1
i < arr.length	compare	N
i++	assign	N - 1
arr[i] > max	access + compare	N - 1
max = arr[i] (worst case)	assign + access	N - 1
return max	return	1

findMax() -- Numbered Lines

Keep this shape in mind

```
public static int findMax(int[] arr) {
    int max = arr[0];                          // (1)
    for (int i = 1; i < arr.length; i++) {     // (2)
        if (arr[i] > max) {                    // (3)
            max = arr[i];                      // (4)
        }
    }
    return max;                                // (5)
}
```

Build T(N) Step By Step

1. (1) max = arr[0] -> 2 ops, once -> 2
2. (2) loop control: init + N tests + (N-1) increments -> 2N
3. (3) arr[i] > max -> 2 ops, (N-1) times -> 2(N-1)
4. (4) max = arr[i], worst case -> 2 ops, (N-1) times -> 2(N-1)
5. (5) return max -> 1 op, once -> 1

```
public static int findMax(int[] arr) {
    int max = arr[0];
    for (int i = 1; i < arr.length; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}
```

Worst Case: T(N) = 6N - 1

$$
\begin{aligned}
T(N) &= 2 + 2N + 2(N-1) + 2(N-1) + 1 \\
&= 2 + 2N + 2N - 2 + 2N - 2 + 1 \\
&= 6N - 1
\end{aligned}
$$

Worst case: 6N - 1

Best case: 4N + 1

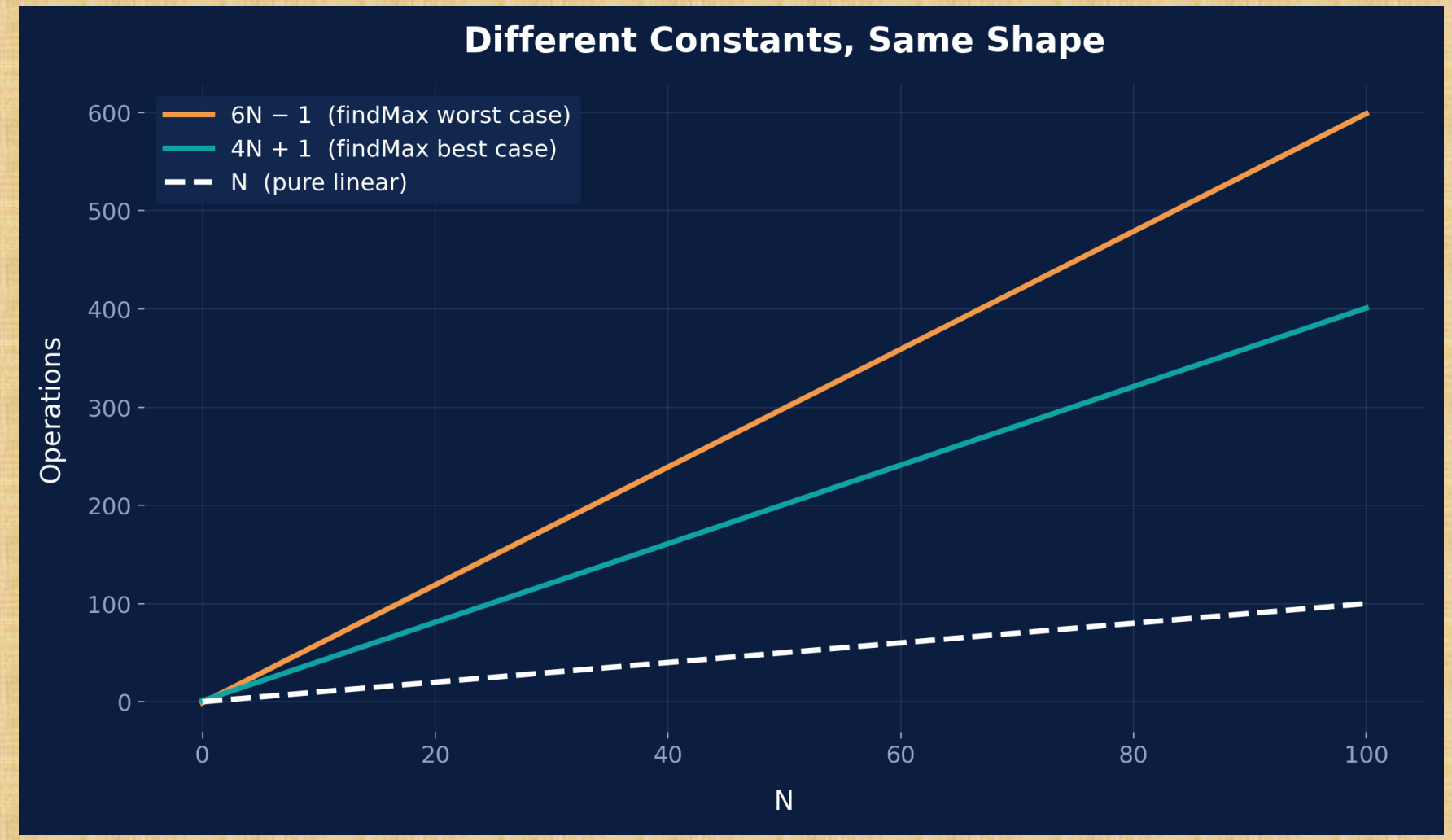
Instrumented findMax()

FindMaxDemo.java

```
int max = arr[0];
counter.countArrayAccess(); counter.countAssignment();

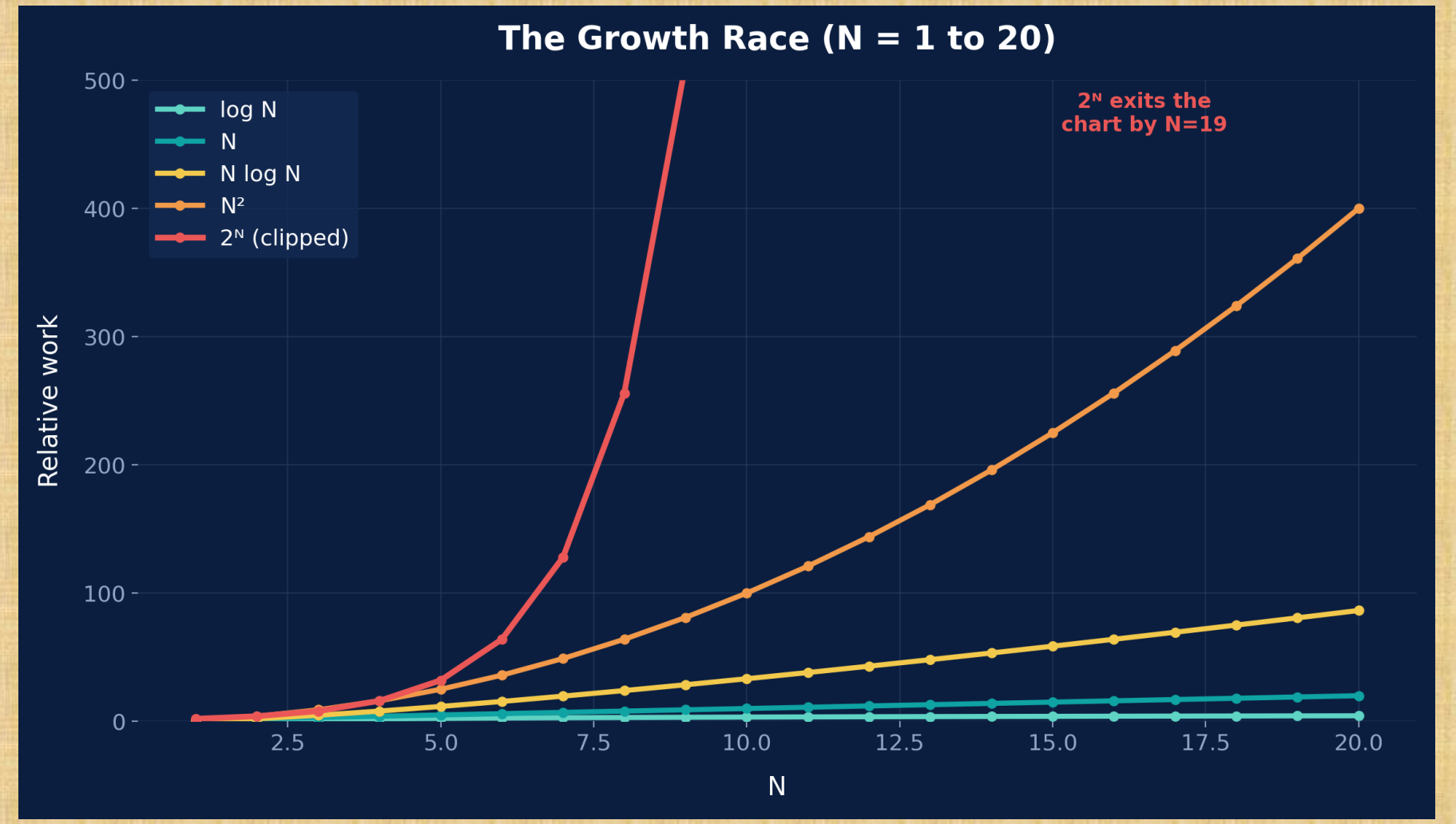
int i = 1; counter.countAssignment();
while (true) {
    counter.countComparison();          // i < arr.length
    if (!(i < arr.length)) break;
    counter.countArrayAccess();
    counter.countComparison();          // arr[i] > max
    if (arr[i] > max) {
        counter.countArrayAccess();
        max = arr[i]; counter.countAssignment();
    }
    i++; counter.countAssignment();
}
counter.countMethodCall(); return max;
// N=100 -> 599 operations (predicted: 6(100)-1 = 599, exact match)
```

Why Constants Don't Matter



If N = 1,000,000, does it really matter whether it's 6N or 4N?

Which Function Wins The Race?



At N = 20, which function already looks like a problem?



INTUITION FIRST

Big-O describes the growth-rate family an algorithm belongs to -- its 'race division,' not its exact speed.

NOW, FORMALLY

The Big-O Definition

```
T(N) is O(f(N)) if there exist positive
constants c and N0 such that:

    T(N) <= c . f(N)    for all N >= N0
```

c = 'the constant that didn't matter'

N0 = 'eventually / for large enough N'

4N + 4 is O(N)

```
Claim: 4N + 4 is O(N)

Choose c = 5, N0 = 4
Check: for all N >= 4,  4N + 4 <= 5N ?
        4 <= N   ->  true when N >= 4   [checkmark]
```

State the tightest Big-O for each (30 sec each):

A

$3N^2 + 2N + 7$

B

$T(N) = 100$

C

$N \log N + N$

Best / Average / Worst Case

linearSearch

```
public static int linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) return i;
    }
    return -1;
}

// Best case:    target is arr[0]           ->  O(1)
// Worst case:   target is last or missing  ->  O(N)
// Average case: target is 'in the middle'  ->  O(N)
```

Three Scenarios, Same Algorithm

Best Case

Target is the very first element.
O(1) -- found immediately.

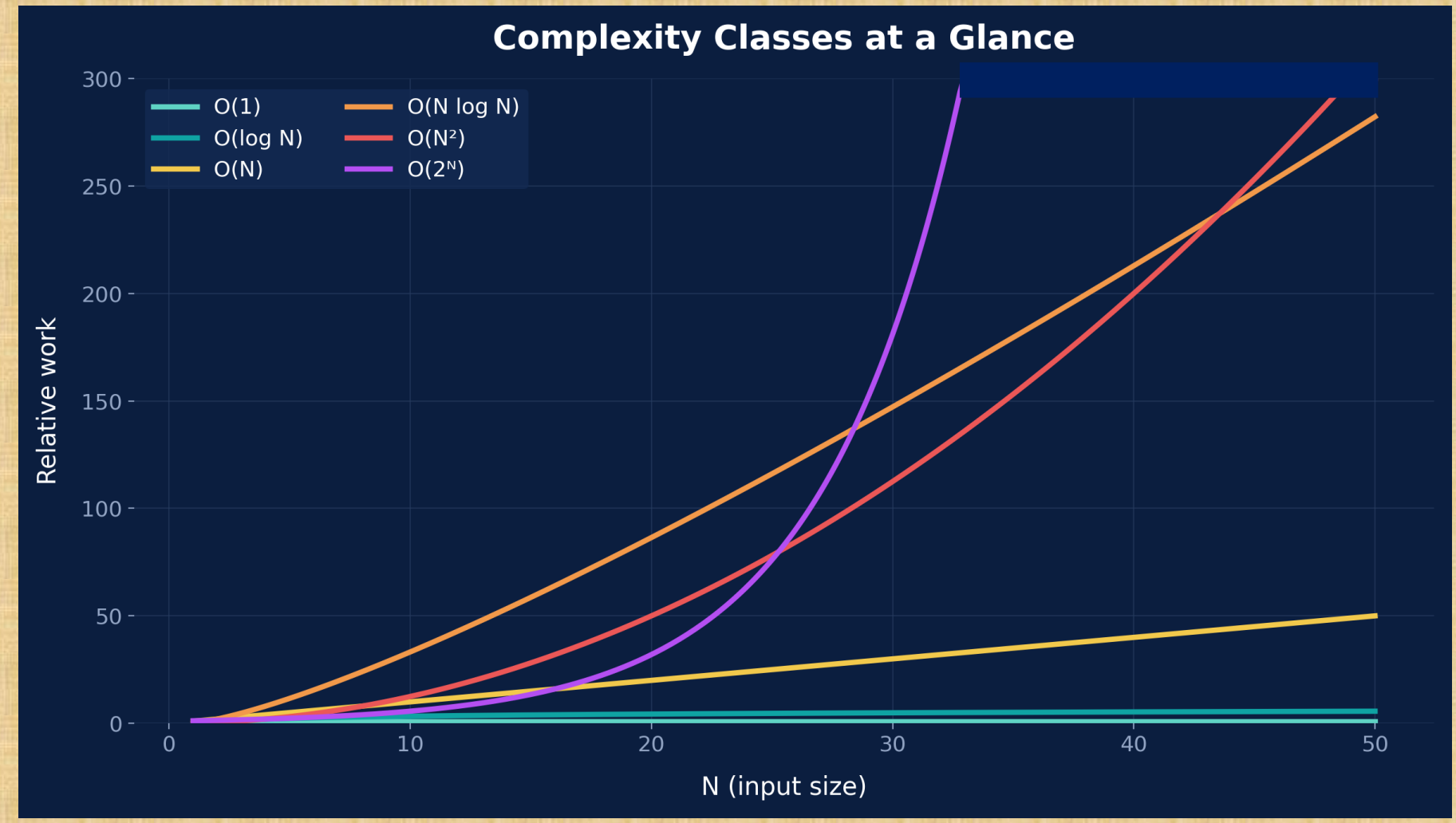
Average Case

Target is somewhere in the middle.
O(N) -- roughly half scanned.

Worst Case

Target is last, or missing entirely.
O(N) -- full scan required.

Complexity Classes At a Glance



This becomes your mental reference for the rest of the course.

Real-Life Analogies

O(1)

Constant

Opening to the first page of a book.

log N

Logarithmic

Guessing a number 1-100 with yes/no hints.

O(N)

Linear

Checking attendance, one name at a time.

N^2

Quadratic

Everyone in the room shaking hands with everyone else.

The Complexity Cheat Sheet

O(1)	**Constant**	Opening the first page	*Array index access*
O(log N)	**Logarithmic**	Guessing a number 1-100	*Binary search*
O(N)	**Linear**	Checking attendance	*Linear search*
O(N log N)	**Linearithmic**	Sorting a playlist efficiently	*Merge sort, Quicksort*
O(N^2)	**Quadratic**	Everyone shaking hands	*Bubble / Selection sort*
O(2^N)	**Exponential**	Trying every subset	*Brute-force subset sum*
O(N!)	**Factorial**	Trying every ordering	*Brute-force traveling salesman*

CAPSTONE ACTIVITY

We Asked an AI a Simple Question

AI

AI Assistant

“Write the fastest search algorithm in Java.”

HERE'S WHAT IT GAVE US

AI's Claim: "O(log n) time -- the fastest search algorithm."

AI-generated

```
public static int search(int[] arr, int target) {
    int low = 0;
    int high = arr.length - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

RECAP

Everything We Covered Today

- ✓ Why analysis matters - Experimental vs mathematical analysis
- ✓ Benchmarking in Java - Limits of benchmarking
- ✓ Counting primitive operations - Deriving T(N)
- ✓ Asymptotic analysis - Best / Average / Worst case
- ✓ Calculating Big-O - Interpreting complexity classes
- ✓ Critically verifying AI-generated code

Engineers in the AI Era

- ✓ AI writes code
- ✓ Engineers verify correctness
- ✓ Engineers analyze complexity
- ✓ Engineers justify design choices
- ✓ Engineers optimize systems

Your value as an engineer isn't typing the code first -- it's knowing whether it's right, fast enough, and worth shipping.